Week 12: Memory &

Knowledge Retrieval in Agents

Short-term vs Long-term · Vector Stores · RAG · Memory Loops · Multi-Agent Memory

Two-Day Programme Overview — Week 12

DAY 1 — From Amnesia to Retrieval (3.5 hrs)	DAY 2 — From Retrieval to Intelligence (3.5 hrs)
BLOCK 1 — Cold Open: The Amnesia Problem (20 min)	BLOCK 6 — Rapid-Fire Recap (15 min)
BLOCK 2 — Memory Types: How Agents Remember (40 min) ★ QUIZ	BLOCK 7 — Retrieval-Augmented Memory Loop (45 min) ★ QUIZ
BLOCK 3 — Short-term vs Long-term Memory (40 min) ★ QUIZ	BLOCK 8 — Multi-Agent Memory Sharing (35 min) ★ QUIZ
BLOCK 4 — Vector Stores: The Long-term Engine (45 min) ★ QUIZ	BLOCK 9 — Case Study: Customer Support with Memory (45 min)
BLOCK 5 — RAG: Retrieval-Augmented Generation (45 min)	BLOCK 10 — Governance, Privacy & Production (25 min)
Buffer / Q&A (10 min)	BLOCK 11 — Full Quiz + Assignments + 5 Laws (40 min)

BLOCK 1

Cold Open — The Amnesia Problem

20 minutes · Hook + Human Memory Model + Why Agents Need Memory

?

Imagine your best employee wakes up every morning with total amnesia. They are brilliant — but every single day, they start from scratch.

That is what your AI agent does right now. Every crew.kickoff() is Day 1 again.

The Amnesia Problem — With vs Without Memory

Agent WITHOUT Memory ✗	Agent WITH Memory ✓
Every interaction starts from zero	Recalls past interactions and outcomes
Repeats the same mistakes every run	Learns from past failures
Cannot reference prior context	Retrieves relevant prior experience
Asks the same questions every time	'I see you called about this on March 5th...'
What most demo agents are today	What production agents MUST be

> *Memory is the difference between a tool you USE and a colleague you WORK WITH. Tools don't remember. Colleagues do.*

Human Memory Architecture → Agent Memory

SENSORY MEMORY → INPUT BUFFER

Raw input, held < 1 second
Agent: the raw user query before processing

WORKING MEMORY → CONTEXT WINDOW

Active processing, limited capacity (7±2 items)
Agent: the LLM's context window (4K-128K tokens)

LONG-TERM MEMORY → VECTOR STORE / DB

Permanent storage, retrieval by association
Agent: FAISS, Chroma, Pinecone — searchable by similarity

Today: we build all three layers for your agents — from context window to persistent vector store

BLOCK 2

Memory Types — How Agents Remember

40 min · 3 Types + LangChain Memory Classes + Real-World + Practice + Quiz

Three Types of Agent Memory

Episodic	Semantic	Procedural
WHAT HAPPENED	**WHAT IS TRUE**	**HOW TO DO**
Example: Customer called on March 5th about billing error. Issue resolved with refund.	Example: Company refund policy: full refund within 30 days, partial after 30 days.	Example: Step 1: verify identity. Step 2: check order. Step 3: process refund.
ConversationBufferMemory — stores full conversation history	VectorStoreRetrieverMemory — retrieves facts by similarity	Agent tools + system prompts — encoded behaviour patterns

REAL WORLD

Semantic Memory at Planetary Scale

Google Search (8.5 billion queries/day)

In the real world

- Google indexes 100+ trillion pages — permanent knowledge
- You query: 'refund policy for damaged goods'
- Google retrieves top-10 most relevant pages by similarity
- It does NOT remember YOUR query history (that's episodic)
- Pure semantic: 'what is true in the world right now?'
- Index updates every few hours — memory is not static

What this maps to in agents

- Your FAISS vector store = mini Google for your agent
- You embed documents into vectors — indexing
- Agent queries: 'what do I know about this topic?'
- Top-k retrieval: return the most relevant chunks
- The vector store IS your agent's semantic memory
- Unlike episodic: no 'when did I learn this' — just 'what do I know'

The lesson: Google Search is a 2-trillion-dollar semantic memory system. You will build the same architecture — at agent scale — in Part 3 of today's notebook.

LangChain Memory Classes — The Building Blocks

```
from langchain.memory import (
    ConversationBufferMemory,     # Stores EVERYTHING (episodic)
    ConversationSummaryMemory,    # Summarises old turns (compressed episodic)
    ConversationBufferWindowMemory,  # Last k turns only (sliding window)
)

# Buffer: remembers all 500 turns. Token cost: HIGH. Context: FULL.
buffer_mem = ConversationBufferMemory()

# Summary: compresses 500 turns to 1 paragraph. Token cost: LOW. Context: LOSSY.
summary_mem = ConversationSummaryMemory(llm=llm)

# Window: keeps last 5 turns. Token cost: FIXED. Context: RECENT ONLY.
window_mem = ConversationBufferWindowMemory(k=5)
```

KEY POINT: Buffer = full history. Summary = compressed history. Window = recent history. Choose based on token budget and context needs.

YOUR TURN

Part 1 — Memory Types in LangChain

12 min

Open: Week12_Teaching_Notebook.ipynb

1. Run Setup cell — confirm LLM loaded
2. Read the 3 memory class definitions (Buffer, Summary, Window)
3. Run the 10-turn customer conversation through BufferMemory
4. Run same conversation through SummaryMemory — compare output
5. Run same conversation through WindowMemory(k=3) — what is lost?

Report back: What did WindowMemory forget that BufferMemory kept? Call on 2 students.

QUICK CHECK

Memory Types Check

1. Name the 3 types of agent memory and give one example of each.
2. Which LangChain memory class keeps the FULL conversation history?
3. Why would you choose SummaryMemory over BufferMemory?

Answer after students respond — click to reveal on next slide

ANSWERS — Memory Types Check

1 3 types of agent memory?

Episodic (what happened — conversation logs), Semantic (what is true — knowledge base), Procedural (how to do — tools + prompts)

2 Which class keeps full conversation?

ConversationBufferMemory — stores every turn, token cost grows linearly

3 Why Summary over Buffer?

Long conversations: Buffer exceeds context window limit. Summary compresses old turns into a paragraph, keeping token count stable.

BLOCK 3

Short-term vs Long-term Memory

40 min · Context Window Limits + Token Economics + Practice + Quiz

Short-term vs Long-term Memory

SHORT-TERM (Context Window)	LONG-TERM (Vector Store / Database)
Lives inside the LLM call (prompt + response)	Lives outside the LLM — persistent storage
Limited: 4K to 128K tokens depending on model	Unlimited: FAISS can hold millions of vectors
Expensive: you PAY per token, every call	Cheap: stored once, retrieved only when needed
Fast: already in the prompt — no retrieval needed	Slower: requires embed → search → retrieve step
Volatile: gone after the API call returns	Persistent: survives restarts, deployments, updates

Production pattern: store everything in long-term memory. Retrieve only what is relevant into the context window. This is RAG.

REAL WORLD

ChatGPT's Context Window IS Short-term Memory

ChatGPT / Claude
(every major chatbot)

In the real world

- Start a new chat: AI knows nothing about you
- Tell it your name and project: it remembers for THIS chat
- Continue for 50 messages: it still remembers (context window)
- Hit 128K tokens: it starts 'forgetting' early messages
- Start a NEW chat: total amnesia — knows nothing again
- ChatGPT Plus 'Memory' feature: stores key facts in long-term DB

What this maps to in agents

- New chat = empty context window = no short-term memory
- Messages accumulate in context window = short-term building
- 128K token limit = working memory capacity (like 7±2 items for humans)
- Forgetting early messages = buffer overflow
- Memory feature = long-term store outside the context window
- Your agent needs BOTH: context window + vector store

The lesson: ChatGPT charges $20/month partly to pay for the context window tokens. Every character you see is being paid for. That is why memory architecture matters.

The Token Economics of Memory

Scenario	Tokens	Cost/call	Monthly (1K/day)
100-turn customer support chat	~50K tokens	$0.075 per call	~$2,250/month for 1K daily calls
Same chat with SummaryMemory	~5K tokens	$0.0075 per call	~$225/month (10x savings)
Same chat with RAG retrieval	~3K tokens	$0.0045 per call	~$135/month (17x savings)

Memory architecture is not just about capability — it is about COST. The right memory type saves thousands per month at scale.

YOUR TURN

Part 2 — Short-term Memory Comparison

12 min

Open: Week12_Teaching_Notebook.ipynb

1. Run a 15-turn conversation using ConversationBufferMemory — count final tokens
2. Run same conversation using ConversationSummaryMemory — compare token count
3. Run same conversation using ConversationBufferWindowMemory(k=5) — compare
4. Ask the agent: 'What did the customer say in turn 2?' — which memory has the answer?
5. Compare: which memory type gives the best answer with the fewest tokens?

Report back: Which memory type had the best quality-to-token ratio? Why?

QUICK CHECK

Short-term vs Long-term Check

1. What happens when a conversation exceeds the LLM's context window?
2. Name one advantage of long-term memory over short-term.
3. Why does RAG use 17x fewer tokens than full buffer memory?

Answer after students respond — click to reveal on next slide

ANSWERS — Short-term vs Long-term Check

1 What happens when conversation exceeds context window?

Oldest messages are dropped (buffer overflow) or the API returns an error — agent loses early context

2 One advantage of long-term over short-term?

Persistent: survives API calls, restarts, and deployments. Cost: stored once, retrieved only when needed.

3 Why does RAG use fewer tokens?

RAG retrieves ONLY the 3-5 most relevant chunks instead of stuffing the entire history into the prompt

BLOCK 4

Vector Stores — The Long-term Memory Engine

45 min · Embeddings + FAISS + Similarity Search + Practice + Quiz

What Is an Embedding?

An embedding converts text into a list of numbers (a vector)

```
# "The patient has chest pain" → [0.23, -0.87, 0.14, 0.56, ..., -0.33]  (1536 numbers)
# "The client reported cardiac discomfort" → [0.21, -0.85, 0.16, 0.54, ..., -0.31]

# These two vectors are VERY CLOSE in 1536-dimensional space.
# Because the MEANING is similar — even though the WORDS are different.
```

```
# "The weather is sunny today" → [0.91, 0.12, -0.76, 0.33, ..., 0.88]
# This vector is FAR from both medical sentences.
# Because the MEANING is completely different.
```

Similar meaning → nearby vectors → cosine similarity is high (close to 1.0)

Different meaning → distant vectors → cosine similarity is low (close to 0.0)

This is how your agent 'understands' that two different questions are about the same topic — without keyword matching.

REAL WORLD

Vector Similarity Powers 600 Million Users

Spotify / Shazam

In the real world

- Spotify embeds every song into a vector (tempo, key, energy, mood)
- Your listening history = a cluster of vectors
- New song vectors near YOUR cluster → 'Discover Weekly' recommendation
- Shazam: records 10-second clip → embeds the audio → finds nearest match
- Both use the same principle: similarity search in vector space
- No keywords. No genre tags. Pure mathematical similarity.

What this maps to in agents

- Your agent embeds documents into vectors (FAISS / OpenAI Embeddings)
- New query is embedded → search for nearest document vectors
- Top-k results returned → injected into the prompt as context
- No keyword matching. Semantic understanding of meaning.
- Same algorithm Spotify uses for songs, your agent uses for knowledge
- FAISS (Facebook AI) runs Spotify's backend search

The lesson: Spotify's entire 'Discover Weekly' feature is a vector similarity search. The same algorithm you will use in Part 3 of today's notebook.

Building a FAISS Vector Store

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# 1. Create the embedding model
embeddings = OpenAIEmbeddings()  # text-embedding-3-small

# 2. Your knowledge base — any text chunks
docs = [
    "Refund policy: full refund within 30 days of purchase.",
    "Shipping takes 3-5 business days for standard delivery.",
    "Premium members get free express shipping.",
]

# 3. Create the vector store (embed + index in one call)
store = FAISS.from_texts(docs, embeddings)

# 4. Query by meaning (not keywords!)
results = store.similarity_search("how long does delivery take?", k=2)
# Returns the 2 most relevant chunks by cosine similarity
```

KEY POINT: FAISS is free, runs locally, no API calls for search. Embedding calls use OpenAI API.

YOUR TURN

Part 3 — Build a FAISS Vector Store

12 min

Open: Week12_Teaching_Notebook.ipynb

1. Run the embedding + FAISS creation cell — 20 text chunks about customer support
2. Query: 'Can I get my money back?' — check the top-3 results
3. Query: 'How do I change my password?' — different topic, different results
4. Try: 'reembolso' (Spanish for refund) — does semantic search find refund docs?
5. Check the similarity scores — what is the distance threshold for 'relevant'?

Report back: Did the Spanish query find the English refund doc? What does that tell you about embeddings?

QUICK CHECK

Vector Store Check

1. What does an embedding convert text into?
2. How does FAISS find relevant documents without keyword matching?
3. Name one real-world product powered by vector similarity search.

Answer after students respond — click to reveal on next slide

ANSWERS — Vector Store Check

1 Embedding converts text into?

A list of numbers (a vector) — typically 1536 dimensions for OpenAI. Similar meaning → similar vectors.

2 How does FAISS find relevant docs?

Cosine similarity between query vector and document vectors — returns the nearest k matches. No keywords needed.

3 Real-world product using vector similarity?

Spotify (song recommendations), Shazam (audio matching), Google (semantic search), Perplexity (RAG)

BLOCK 5

RAG — Retrieval-Augmented Generation

45 min · The RAG Pipeline + Why RAG Beats Fine-tuning + Practice + Discussion

The RAG Pipeline — 4 Steps

1. QUERY

User asks a question or agent needs context

→

2. RETRIEVE

Search vector store for top-k relevant chunks

→

3. AUGMENT

Inject retrieved chunks into the LLM prompt

→

4. GENERATE

LLM answers using retrieved context + its own knowledge

RAG = don't ask the LLM to memorise everything. Just give it the right documents at the right time.

This is how Perplexity AI, Google AI Overviews, and enterprise AI assistants work.

REAL WORLD

RAG as a $3 Billion Product

Perplexity AI (valued at $3B+, 2024)

In the real world

- User asks: 'What are the side effects of metformin?'
- Perplexity searches the web in real time (retrieve)
- Pulls top-5 most relevant medical sources
- Injects source text into the LLM prompt (augment)
- LLM generates answer WITH citations (generate)
- Result: AI answer backed by real, current sources

What this maps to in agents

- Query = user's clinical question
- Retrieve = FAISS/vector search against your knowledge base
- Augment = inject retrieved docs into agent prompt
- Generate = LLM produces grounded answer
- Citations = expected_output includes source reference
- Your agent can do this with 10 lines of LangChain code

The lesson: Perplexity built a $3B company on RAG. The same pipeline you build in Part 4 is their core product.

RAG vs Fine-tuning — When to Use Each

RAG (Retrieval-Augmented Generation)	FINE-TUNING (retrain the model)
Knowledge changes frequently — just update the docs	Knowledge is static and domain-specific
No model training needed — deploy in hours	Requires training data + GPU time (days/weeks)
Answer includes source citations — auditable	No citations — knowledge baked into weights
Cost: embedding + search (cheap per query)	Cost: $100-$10,000+ per training run
Best for: company docs, policies, product info	Best for: tone/style, domain jargon, formats

Rule of thumb: if the information changes more than once a month, use RAG. If the model needs to BEHAVE differently (not just KNOW differently), consider fine-tuning.

YOUR TURN

Part 4 — Build a RAG Pipeline

15 min

Open: Week12_Teaching_Notebook.ipynb

1. Read the 20 customer support text chunks (knowledge base)
2. Build a FAISS vector store from those chunks
3. Create a RetrievalQA chain using LangChain
4. Ask: 'What is the refund policy for damaged items?'
5. Ask: 'Can I return a gift?' — does it find the right chunk?

Report back: Did the RAG chain cite the correct document chunk? Was the answer grounded or hallucinated?

DISCUSSION QD1

When does RAG fail? Give a scenario where retrieval-augmented generation would give a

Expected Insights (instructor only)

- Retrieved chunk is outdated — policy changed but docs not updated
- Chunk is too small — loses context from surrounding paragraphs
- Top-k returns irrelevant docs — query is ambiguous, matches wrong topic
- Embedding model doesn't understand domain jargon — medical terms embed poorly
- Hallucination on top of retrieval — LLM extrapolates beyond what the chunk says

What You Built Today

- Human memory model → agent memory architecture (3 layers)
- 3 memory types: Episodic, Semantic, Procedural — when to use each
- LangChain memory classes: Buffer, Summary, Window — with token cost comparison
- FAISS vector store — your agent's long-term memory engine (Spotify's algorithm)
- RAG pipeline — retrieve relevant docs, augment the prompt, generate grounded answers (Perplexity's product)

Tomorrow: Memory Loops · Multi-Agent Shared Memory · Customer Support Case Study · Governance · Full Quiz

Week 12 · Day 2

From Retrieval to Intelligence

Memory Loops · Multi-Agent Memory · Customer Support Case · Governance · Assignments

DAY 2 of 2

RAPID FIRE RECAP — Day 2 Opener

No slides — answer from memory!

Q1. Name the 3 types of agent memory.

Q2. What is the difference between short-term and long-term memory in agents?

Q3. What does an embedding convert text into?

Q4. Name the 4 steps of the RAG pipeline.

Q5. Why does RAG use fewer tokens than full buffer memory?

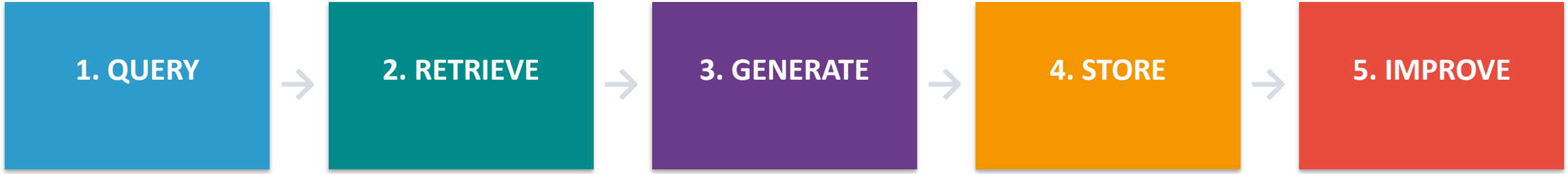
Q6. What real-world product is built entirely on RAG?

BLOCK 7

Retrieval-Augmented Memory Loop

45 min · Full Loop + Memory Decay + Practice + Quiz

The Memory Loop — 5 Steps



↩ **STORE feeds back into RETRIEVE — each cycle makes retrieval BETTER**

RAG stops at step 3 (generate). The Memory Loop adds steps 4-5.
Step 4: STORE the new interaction (query + answer + outcome) back into the vector store.
Step 5: IMPROVE — rank stored memories by relevance, decay old ones, consolidate similar ones.
Result: agent gets BETTER over time. Each interaction makes the retrieval more relevant.

REAL WORLD

Netflix — The Memory Loop Running 250 Million Times

Netflix Recommendation Engine

In the real world

- User watches a movie → STORE viewing event + completion %
- Next visit → RETRIEVE viewing history to personalize homepage
- GENERATE: rank 15,000 titles for THIS user specifically
- STORE: did user click? Did they finish? New data point stored
- IMPROVE: decay old preferences (you liked romcoms 3 years ago)
- 250M users × continuous loop = system gets smarter every second

What this maps to in agents

- STORE = add interaction to vector store after each agent run
- RETRIEVE = search stored interactions for relevant context
- GENERATE = agent produces answer using retrieved context
- DECAY = old memories get lower relevance scores over time
- IMPROVE = consolidate frequent patterns into compact representations
- Your agent's memory loop IS Netflix's recommendation loop at 1:1 scale

The lesson: Netflix earns $33B/year. 80% of content watched comes from the memory loop (recommendations). That is the business value of a well-built retrieval-augmented memory system.

Memory Decay — Why Agents Must Forget

- **Without decay: memory grows forever**

  Retrieval quality degrades — irrelevant old memories compete with relevant new ones

- **Without decay: token costs increase**

  More stored memories = more embeddings = higher search cost per query

- **Decay strategies:**

- **Time-based decay**

  Reduce relevance score by X% per day. Old memories gradually fade.

- **Usage-based decay**

  Memories that are never retrieved lose relevance. 'If you never recall it, you don't need it.'

- **Consolidation**

  Merge 100 similar old memories into 1 summary. Compress, don't delete.

> *Netflix decays your viewing history. Google decays search index entries. Your agent must decay old memories too — or it drowns in noise.*

YOUR TURN

Part 5 — Memory Loop with Decay

15 min

Open: Week12_Teaching_Notebook.ipynb

1. Run the MemoryLoopAgent — handles 5 customer interactions sequentially
2. After each interaction: agent stores the conversation in FAISS
3. Query 6 references a topic from query 1 — does the agent remember?
4. Enable decay: set decay_rate=0.1 and re-run
5. Query 6 again: does the agent still retrieve query 1? Compare scores.

Report back: How did decay_rate affect retrieval of the oldest memory? Was it still found?

QUICK CHECK

Memory Loop Check

1. What 2 steps does the Memory Loop add beyond RAG?
2. Why must agents have memory decay?
3. How does Netflix's recommendation engine use a memory loop?

Answer after students respond — click to reveal on next slide

ANSWERS — Memory Loop Check

1 2 steps beyond RAG?

Step 4: STORE the interaction back into the vector store. Step 5: IMPROVE — decay old memories, consolidate, rank.

2 Why memory decay?

Without decay: old irrelevant memories compete with recent relevant ones, degrading retrieval quality and increasing cost.

3 Netflix memory loop?

Watch event stored → next visit retrieves viewing history → personalised ranking generated → new interaction stored → old preferences decayed

BLOCK 8

Multi-Agent Memory Sharing

35 min · Shared vs Private Memory + Practice + Quiz

Shared vs Private Agent Memory

SHARED MEMORY	PRIVATE MEMORY
All agents read/write to the same store	Each agent has its own memory store
Best for: team knowledge, company policies	Best for: personal context, user preferences
Risk: one agent's bad data pollutes everyone	Safe: one agent's memory cannot corrupt another
Example: hospital medical records (all clinicians read)	Example: each customer support agent remembers their own calls

Production pattern: SHARED knowledge base (policies, facts) + PRIVATE interaction memory (user history, preferences). Never mix the two.

REAL WORLD

Shared Memory in Healthcare — Every Clinician Reads the Sa

Hospital Electronic Medical Records (EMR)

In the real world

- Patient arrives → triage nurse records vitals and symptoms
- ER doctor reads the SAME record — adds diagnosis
- Lab tech reads the SAME record — runs ordered tests
- Pharmacist reads the SAME record — checks drug interactions
- All 4 clinicians share ONE memory store for this patient
- Private notes: each clinician keeps their own reasoning separate

What this maps to in agents

- Shared FAISS store = EMR. All agents embed docs into it
- triage_agent writes patient symptoms → shared store
- planning_agent reads shared store → generates care plan
- monitoring_agent reads shared store → checks for conflicts
- Private memory: each agent's task history stays separate
- Architecture: shared knowledge + private episodic memory

The lesson: A hospital where clinicians cannot read each other's notes kills patients. Agents without shared memory make the same mistake — in less dramatic but equally costly ways.

YOUR TURN

Part 6 — Multi-Agent Shared Memory

12 min

Open: Week12_Teaching_Notebook.ipynb

1. Create a shared FAISS vector store
2. Agent 1 (researcher) writes 5 findings into the shared store
3. Agent 2 (analyst) queries the shared store — can it find Agent 1's findings?
4. Agent 2 adds its OWN analysis back to the store
5. Agent 1 queries again — can it see Agent 2's analysis? (bidirectional sharing)

Report back: Did Agent 2 successfully retrieve Agent 1's findings without being told about them directly?

QUICK CHECK

Multi-Agent Memory Check

1. When should agents share memory vs keep private memory?
2. What is the risk of ALL agents writing to one shared store?
3. Name a real-world system that uses shared memory across multiple agents.

Answer after students respond — click to reveal on next slide

ANSWERS — Multi-Agent Memory Check

1 Shared vs private?

Shared: company policies, patient records, knowledge base facts. Private: user preferences, agent task history, reasoning notes.

2 Risk of shared store?

Memory pollution: one agent writes incorrect data → all other agents retrieve and act on it. Like a wrong allergy note in a medical record.

3 Real-world shared memory?

Hospital EMR, corporate wikis, Amazon product catalog (shared by all recommendation agents), Google Knowledge Graph

BLOCK 9

Case Study: Customer Support with Memory

45 min · Full Build + Before/After Comparison + Practice

The Problem — Customer Calls 3 Times About the Same Issue

WITHOUT Memory ✗

Call 1: 'I was charged twice for my subscription.' → Agent asks for account number, investigates, says refund takes 5 days.

Call 2 (next day): 'Has my refund been processed?' → Agent asks for account number AGAIN. No record of Call 1. Starts from scratch.

Call 3 (5 days later): 'I still haven't received my refund.' → Agent asks for account number AGAIN. Customer is furious.

WITH Memory ✓

Call 2: 'I see you called yesterday about a double charge on your subscription. Your refund of $29.99 was initiated. It takes 5 business days.'

Call 3: 'Your refund of $29.99 was processed on Day 3. It should be in your account by now. Can you check and confirm?'

Same agent. Same LLM. Same code. The only difference: memory.

REAL WORLD

Customer Memory — The $100B Support Industry Standard

Zendesk / Intercom / Amazon Support

In the real world

- Zendesk stores every customer ticket, call, and chat — forever
- Agent opens ticket: sees full history before saying a word
- 'I see you contacted us 3 times about order #4521'
- Amazon: 'We see you recently ordered [product]. Is this about that?'
- Customer satisfaction jumps 30-40% with context-aware support
- Without memory: CSAT drops, handle time doubles, escalation triples

What this maps to in agents

- Ticket history = episodic memory in the vector store
- Agent reads history before responding = RAG retrieval step
- 'I see you contacted us...' = retrieved context in the prompt
- Each new interaction stored = memory loop closes
- CSAT improvement = measurable business value of memory
- Your Part 7 notebook builds exactly this system

The lesson: Zendesk's entire business model is giving support agents memory. Without it, every interaction is a cold start — and customers leave.

YOUR TURN

Part 7 — Customer Support Agent with Memory

15 min

Open: Week12_Teaching_Notebook.ipynb

1. Read the MemoryCustomerSupportAgent class — note the retrieve + respond + store loop
2. Run Call 1: 'I was charged twice for my subscription' — agent responds + stores interaction
3. Run Call 2: 'Has my refund been processed?' — does the agent recall Call 1?
4. Run Call 3: 'I still haven't got my refund' — does it reference the timeline?
5. Try: Run Call 1 with a DIFFERENT customer name — does it correctly separate memories?

Report back: Did the agent correctly recall Call 1 during Call 2 without being told? Did it separate different customers?

BLOCK 10

Governance, Privacy & Production

25 min · What to Remember + What to Forget + GDPR + HIPAA + Discussion

Memory Governance — 5 Questions for Production

Block 10

- **1. What SHOULD the agent remember?**

  Task-relevant facts, customer context, outcomes — not personal opinions or raw PII

- **2. What MUST the agent forget?**

  GDPR Article 17: 'Right to Erasure'. Customer says 'delete my data' → all their memories must be deletable

- **3. Who controls the memory?**

  The organisation, not the agent. Memory policies must be designed by humans, enforced by code

- **4. How do you prevent memory poisoning?**

  Validate before storing. Do not let agents store user-injected text directly into shared memory

- **5. How do you audit what the agent remembers?**

  Every stored memory needs a timestamp, source, and deletion mechanism — full audit trail

> *GDPR Right to Erasure + HIPAA data minimisation = your agent's memory system MUST support selective deletion. If it cannot delete, it is not compliant.*

REAL WORLD

Memory Misuse — What Happens Without Governance

Cambridge Analytica (2018 Scandal)

In the real world

- Facebook collected user data (likes, shares, friend lists)
- Cambridge Analytica accessed 87 million profiles without consent
- Used stored data to build psychological profiles
- Targeted political ads based on stored personality traits
- Result: $5B FTC fine, CEO congressional testimony, GDPR acceleration
- Core issue: data was STORED without governance on who could ACCESS it

What this maps to in agents

- Your agent's vector store = a database of user interactions
- Without access controls: any process can read stored memories
- Without deletion capability: you cannot comply with 'delete my data'
- Without audit trail: you cannot prove what was stored or accessed
- Memory governance IS data governance — same rules apply
- GDPR / HIPAA are not optional features — they are legal requirements

The lesson: Cambridge Analytica proved that stored data without governance is a liability, not an asset. Your agent's memory system is the same.

DISCUSSION QD2

For a clinical AI agent with patient memory: what MUST it remember, what MUST it forget, and

Expected Insights (instructor only)

- Must remember: allergies, current medications, treatment history — safety-critical
- Must forget: patient requests deletion (GDPR), stale information (discontinued medications)
- Who decides: clinicians set memory policy, not engineers — domain expertise required
- Audit trail: every memory write must log who/what/when for compliance
- HIPAA: minimum necessary data only — do not store more than the task requires

BLOCK 11

Full Assessment + 5 Laws of Agent Memory

40 min · Quiz + Assignments + Closing

QUIZ

Memory & Knowledge Retrieval — Week 12 (1/2)

1. **Name the 3 types of agent memory and one LangChain class for each.**
2. **What is the difference between ConversationBufferMemory and ConversationSummaryMemory?**
3. **What does an embedding convert text into, and why is this useful?**
4. **Name the 4 steps of the RAG pipeline.**
5. **Why does RAG use fewer tokens than sending the full conversation history?**

   a) RAG compresses the LLM b) RAG retrieves only relevant chunks c) RAG skips the prompt d) RAG uses a smaller model

QUIZ

Memory & Knowledge Retrieval — Week 12 (2/2)

6 What 2 steps does the Memory Loop add beyond basic RAG?

7 Why must agents have memory decay? Give one consequence of not having it.

8 What is the difference between shared memory and private memory in multi-agent systems?

9 Under GDPR, what must an agent's memory system support when a user says 'delete my data'?

1 What is memory poisoning and how do you prevent it?

ANSWER KEY — Week 12 Answer Key (Q1–Q5)

1 Episodic (ConversationBufferMemory), Semantic (VectorStoreRetrieverMemory / FAISS), Procedural (Agent tools + system prompts)

2 Buffer stores EVERY turn (high token cost, full context). Summary compresses history into a paragraph (low token cost, lossy context).

3 A vector (list of 1536 numbers). Similar meaning → nearby vectors → enables semantic search without keyword matching.

4 1. Query → 2. Retrieve relevant chunks from vector store → 3. Augment the prompt with retrieved chunks → 4. Generate answer using LLM

5 b) RAG retrieves only the top-k most relevant chunks instead of stuffing everything into the prompt

ANSWER KEY — Week 12 Answer Key (Q6–Q10)

6 Step 4: STORE the interaction back into the vector store. Step 5: IMPROVE — decay old memories, consolidate, re-rank.

7 Without decay: old irrelevant memories compete with recent relevant ones, degrading retrieval quality. Example: 2-year-old support data about discontinued products polluting current results.

8 Shared: all agents read/write same store (company knowledge). Private: each agent has own store (user preferences, task history). Never mix them.

9 Selective deletion — must be able to find and remove ALL memories associated with that specific user. GDPR Article 17 'Right to Erasure'.

1 Memory poisoning: malicious user tricks agent into storing false information in shared memory, corrupting all future retrievals. Prevention: validate before storing, never store raw user input directly.

Practice Assignment — 3 Levels (present at next session)

LEVEL 1 — Foundational	LEVEL 2 — Intermediate	LEVEL 3 — Advanced
Build a Q&A chatbot that uses ConversationBufferMemory. Test with 10 turns. Then switch to SummaryMemory and compare: (1) token count difference, (2) answer quality difference, (3) which is better for this use case and WHY.	Build a RAG pipeline over a knowledge base of 50+ text chunks about your domain. Implement memory decay: after each query, reduce relevance of chunks not retrieved by 5%. Run 20 queries and show how retrieval results change over time.	Build a multi-agent customer support system with SHARED knowledge base (policies) + PRIVATE per-customer memory (interaction history). Support 3 different customers. Implement selective deletion for one customer. Prove the other customers' memories are unaffected.

Every Production Agent Memory System Needs All Five

If your system is missing any of these — it is not production-ready.

1. PERSISTENCE

Memory must survive restarts. If it lives only in the context window, it is not memory — it is a chat log.

2. RETRIEVAL

Store is useless without search. Every memory must be retrievable by semantic similarity, not just ID lookup.

3. DECAY

Agents must forget. Without decay, memory becomes noise. Netflix decays. Google decays. Your agent must too.

4. SEPARATION

Shared knowledge + private history. Never mix company facts with user interactions in one unscoped store.

5. GOVERNANCE

Who can read, write, delete? Every memory needs an audit trail. GDPR and HIPAA are not features — they are legal requirements.

Quick Reference — Real-World Anchors (share as handout)

AI Concept	Real-World Anchor	One-Liner
Short-term memory	ChatGPT context window	Limited, expensive, volatile
Long-term memory	FAISS / vector store	Persistent, cheap, searchable
Embeddings	Spotify song vectors	Meaning → numbers → similarity
RAG pipeline	Perplexity AI	Retrieve → augment → generate
Memory loop	Netflix recommendations	Retrieve → answer → store → improve
Shared memory	Hospital medical records (EMR)	All agents read the same facts
Memory decay	Netflix preference aging	Old memories lose relevance over time
Memory governance	Cambridge Analytica lesson	Store without governance = liability

Resources & Next Steps

- **Notebooks — OneDrive_1_4-3-2026/ folder**

  Week12_Teaching_Notebook.ipynb · Reinforcement_Loop_Simple.ipynb · All Week 11 notebooks

- **Key Libraries**

  LangChain Memory: langchain.memory · FAISS: faiss-cpu · OpenAI Embeddings: langchain_openai

- **Further Reading**

  LangChain Memory docs · FAISS GitHub (facebook/faiss) · Perplexity AI blog on RAG architecture

- **Practice**

  Build a RAG agent over YOUR company's documentation. Add memory decay. Add GDPR deletion. That is the assignment at scale.

> *The best way to learn agent memory is to build an agent that remembers badly — then fix it.*